

When Your LLM Is Wrong, What Kind of Wrong Is It?

A diagnostic ladder — structure, meaning, context, groundedness, environment — for the conversation between engineers and the people who use what they build.

Lodewijk Bonebakker*

April 2026

A junior developer on your team has just shipped a small change. They asked the language model to write a function that filters customer records by region and returns the names sorted by purchase total. The function passed code review. It runs without error. The output is wrong, and nobody can quite explain why.

This is, by now, a familiar experience for engineering teams using LLMs in production. The model produces something that looks right. It is wrong in ways that are hard to articulate. The team falls back on the practitioner vocabulary that has accumulated over the last five years: someone says *hallucination*, someone else says *prompt brittleness*, a third person mentions *lost in the middle*. Each of these terms picks out a real phenomenon. None of them, considered as a set, forms a coherent diagnostic procedure.

The vocabulary the field has built for itself is sharp at the level of mechanism (cross-entropy training optimizes plausibility, not truth) and dull at the level of human experience (the answer just felt off). The gap between these two registers is where most production incidents are diagnosed badly, communicated unclearly, or fixed in the wrong place. This article proposes a closing of that gap: a five-axis model of LLM error, ordered from the most mechanical (what the model is best at) to the most human (what only the user can supply), that practitioners can hold in their heads and walk through quickly.

The five axes are *structure*, *meaning*, *context*, *groundedness*, and *environment*. They form a ladder. At the bottom, the model is at its strongest and failures are rare. At the top, the model is, in a precise sense, at the mercy of the human, and failures are inevitable unless the human compensates. The model is not a theory. It is a vocabulary, with a procedure attached, designed to be useful in conversations with stakeholders who do not have a year to spend on neural-network internals. The full failure landscape has two parts: the five-axis failures that the ladder addresses directly, and four additional classes that escape the single-output frame entirely. This article maps both, then shows the ladder in action.

Why a layered model, and why now

The instinct to organize language around layers is older than the language model. Linguists since the mid-twentieth century have distinguished syntax (the rules of well-formedness), semantics (what an utterance means), and pragmatics (the situational effect it has). Each layer has its own rules and its own diagnostic vocabulary. When we ask whether a sentence makes sense, we are asking different questions at each level, and we know which questions belong on which level.

Two strands of recent work make this organization newly relevant. Stevan Harnad's 1990 paper *The Symbol Grounding Problem* [1] asked how the symbols inside a computer could

*Drafting assistance: Claude Opus 4.7 (Anthropic). The author chose the framework, scope, and voice; the model produced prose under direction; the author edited. Errors that survived this process are the author's.

come to refer to anything outside the computer at all. Emily Bender and Alexander Koller’s 2020 paper *Climbing towards NLU* [2] sharpened the question: a system trained only on linguistic form cannot, by that training alone, learn meaning, because meaning is a relation between form and the world that the form alone does not contain. The position has not gone unchallenged — subsequent work has argued that distributional structure carries more semantic information than a strict form-only argument allows — but the form-meaning gap is real enough for the purposes of this article. The grounding problem is older and more general than any particular training objective. It manifests in current systems through the cross-entropy loss they are trained on: the loss rewards the plausibility of the next token, not its truth. The practitioner literature on hallucination [3] traces several distinct causes — data noise, decoding stochasticity, mis-calibration, and exposure bias as well as the underlying form-meaning gap — and the part of that taxonomy this article concerns itself with is the form-meaning gap.

The five-axis model proposed here is an attempt to operationalize the grounding distinction in a way practitioners can use. It places structure and meaning on the model side, environment on the human side, and context and groundedness in between. The boundary between what the model can do and what the human must supply turns out to be the most useful division to teach.

Structure: is the output well-formed?

A model’s output is structurally correct if it satisfies the formal constraints of the format being requested. A JSON object that parses; a Python script that imports; a SQL query the database accepts; a LaTeX document that compiles. Structure is the most mechanical of the five axes and is, for that reason, where modern LLMs perform best.

A structural failure is the kind a parser can catch. The fix lives in the decoding stack — JSON-mode, schema-validated sampling, regex-constrained generation, finite-state-machine guards over the token distribution. In current practice, frontier models handle short, well-specified formats reliably; engineering teams who still have to debug structural failures regularly are usually doing something more demanding: deeply nested formats, formats outside the training distribution, long-running structured-output streams, or multi-step tool-call traces.

The diagnostic risk on this axis is the inverse of the failure rate. Structural correctness is so easy to verify that practitioners over-rely on it. A syntactically valid JSON object can still mean the wrong thing, say the wrong thing, and refer to a thing that does not exist. A practitioner who confirms structure and stops has confirmed the least interesting property of the output.

Meaning: is the output internally coherent?

A structurally correct output can fail at the level of internal coherence. The model produces a paragraph whose first sentence contradicts its third. The function it writes declares a return type of `list[str]` but actually returns tuples. The argument relies on a premise the same output has elsewhere denied. These failures are detectable by careful reading without any appeal to the world; they are failures of the output considered as a self-contained artifact.

The mechanism is the same one that produces fluent text in the first place. A language model trained to predict the next token given the prefix optimizes for the plausibility of each token

in isolation. Plausibility is local. A token can be plausible at every position in a sequence and still produce, at the end, a global contradiction. A single forward pass has no architectural verifier; the model emits each token conditioned on the prefix without checking the output as a whole against itself, and coherence is a side effect of training rather than an enforced invariant.

Recent training narrows the gap. Reasoning-trained models — the OpenAI o1/o3 line, DeepSeek R1, Claude with extended thinking — learn to walk back contradictions in their own reasoning traces during generation, and inference-time techniques (self-consistency sampling, self-refine, separate verifier passes, agentic critique loops) reduce inconsistency further at the cost of compute and orchestration complexity. None of these is a guarantee in the way a JSON parser is for structure, but they materially shift the failure-rate curves and should not be ignored when stakes are high.

The axis is, even so, one that benchmarks tend to overstate. A model that scores 85 on a multiple-choice reading-comprehension test may produce, in free generation, paragraphs that contradict themselves in subtle ways. Domain expertise unmask this: the technical reader who knows the territory catches inconsistencies the casual reader does not. The diagnostic discipline is to read the output as a self-contained artifact and to know which inference-time mitigation to engage when the cost of a meaning failure justifies the additional compute.

Context: is the output appropriate for the situation?

A model can produce a structurally valid, internally coherent paragraph that nevertheless answers the wrong question. A user asks for a summary of a research paper for a non-specialist audience and receives a paragraph dense with jargon. A user asks a quick yes-or-no question and receives a five-paragraph essay. A user asks a clarifying question and receives a generic safety disclaimer. None of these are factually wrong. All of them are wrong for the situation.

The mechanism is the gap between what the user knows and what the prompt encodes. The model has access only to the tokens in its context window. Anything the user knows about themselves, their audience, or their goals that is not written into the prompt does not exist for the model. Few-shot prompting attempts to encode situational expectations through examples. System prompts attempt to fix the situation up front. Alignment training shapes defaults — politeness, refusal, length, formality — but defaults are defaults, not the situation.

Context failures are easy to recognize and harder to fix. The user can usually point at the answer and say what is wrong with it (“too long,” “too technical,” “misses the actual question”). The fix is almost always to give the model more of the situation: who you are, who you are writing for, why you need the answer, what would count as a good one. The asymmetry between the ease of recognition and the difficulty of fix is the diagnostic signature of a context failure.

Groundedness: does the output connect to reality?

The fourth axis is whether the output is *grounded* — whether its claims connect to verifiable reality, or whether they have drifted into plausible-sounding fabrication. We use *groundedness* for the property the output either has or does not. *Grounding*, in the philosophical literature, is the name of the underlying problem: how symbols inside a system can come to refer to

things outside it at all. The practitioner term in the neighbourhood is *hallucination*, which the surveys [3] split into *factuality* (the output contradicts the world) and *faithfulness* (the output contradicts a source the model was given). Both are groundedness failures in the sense used here.

Groundedness failures are especially hard to catch because no internal reading of the output reveals them. A hallucinated citation has the same structural form as a real one. A hallucinated statistic has the same surface plausibility as a true one. The check must come from outside the text. The model has produced a high-probability continuation of its prompt; that continuation is sometimes true, sometimes false, and the model has no reliable mechanism to tell the two apart on its own. Calibration research has shown the internal signal is non-zero — models do produce confidence estimates that correlate with truth — but the signal is not strong enough to substitute for external verification.

This is the place where the symbol-grounding literature is most directly useful. Harnad’s question — how can symbols mean anything? — remains unresolved for text-only next-token models in isolation. In practice, system design addresses this limitation by linking models to external evidence and tools. The dominant remedy at the system level is retrieval: replace the question “what is the most probable thing to say” with “what is the most probable thing to say given these specific sources.” Most of the grounding burden then shifts outside the model, to the retriever and to the prompt — though residual model-side work remains: the model still has to read the retrieved evidence faithfully, defer to it when its parametric prior conflicts, and reconcile contradictory sources. The RAG-faithfulness literature catalogues the failures that arise when it does not. Tool use extends the same idea to verification: a model that can call a calculator delegates arithmetic; a model that can query a database delegates fact lookup. The pattern is to take as much of the work that demands grounding as possible and move it outside the language model.

The difficulty is compounded by a calibration problem. The model’s expressed confidence — the apparent certainty of its prose — is not a reliable signal of actual reliability. A hallucinated statistic arrives in the same register as a verified one. A fabricated API call sounds as authoritative as a real one. Token-level probabilities correlate with accuracy well above chance, but the correlation is not strong enough to substitute for external verification, and it is inaccessible to users who see prose rather than logits. The practical consequence is that the model’s apparent confidence cannot be used to triage which claims need checking. All claims need checking uniformly, or none of them can be trusted — and the discipline of treating any factual claim as a hypothesis holds across the board, not selectively for claims that happen to sound uncertain.

The diagnostic discipline is to treat any factual claim by the model as a hypothesis. If the claim matters, verify it. If it cannot be verified, do not use it. The careful practitioner builds workflows that make verification cheap (retrieval, citations, linked sources) so that groundedness failures are catchable by construction.

When groundedness failures appear consistently in a specific domain, register, or deployment context, the root cause may be distributional: the model’s fine-tuning distribution is misaligned with the deployment distribution. Retrieval and tool use reduce individual groundedness errors but do not address the underlying mismatch. The systemic remedy is targeted evaluation on representative examples from the actual deployment distribution, not per-call augmentation.

Environment: did the prompt encode the situation?

The fifth axis sits at the human end of the ladder, and is the hardest to see because, by construction, it is invisible. An environment failure occurs when the answer is wrong because the prompt did not contain enough of the situation for the question to be answerable. The user knows things — about themselves, their domain, their constraints, the local conventions of their organization, the events of yesterday afternoon — that the model does not. If the user does not write those things into the prompt, the model cannot use them. The model can produce a fluent, internally coherent, grounded, well-formed answer that is wrong because the question itself was underspecified.

This is the failure mode that experienced LLM users learn to recognize first and explain to others last. From the inside it looks like a model failure: the answer is wrong, so the model must be wrong. From the outside it is a prompt failure: the model gave the most reasonable answer to the question it was actually asked, which differs from the question the user thought they were asking. The gap between the two is the user's environment.

The asymmetry is fundamental; the failure mode it produces is only partially fixable by training. Training can teach the model to ask clarifying questions or abstain when prompts look underspecified, but it cannot conjure information the user never wrote down. A language model has access to its weights and its context window; it does not have access to the user's calendar, the user's team, the user's organization's coding conventions, the user's medical history, the user's project's constraints, or any of the several hundred other things that shape what counts as a good answer in a real situation. Some of these can be imported into the context — through retrieval, through tool use, through careful prompting, through long-running agent contexts that accumulate state. None of them can be imported automatically.

The diagnostic signature is distinctive. The user reads the answer and feels frustrated; the answer is, on its own terms, fine; what is wrong is that it does not address the real situation. The fix is rarely to ask the model again. The fix is to write down, explicitly, the parts of the environment the model needed to see. Practitioners who become good at LLMs are practitioners who have learned which parts of their environment need importing for which kinds of question.

Beyond the five axes

The five axes map the full range of single-output failures — everything that can go wrong when a specific output is in front of you and you are examining it. Six additional failure classes exist alongside them; they escape the single-output frame entirely and require different detection and remediation. The ladder addresses only the five axes. Knowing what lies beyond them before walking makes the ladder's scope explicit and its limits visible.

The first is *normative*. An output that is structurally fine, internally coherent, situationally appropriate, factually grounded, and environmentally informed but is, by some standard the user holds, simply wrong — offensive, biased, in conflict with values the user expects the system to hold. The five axes ask whether the answer is correct. The normative dimension asks whether the answer is right, and right is not the same thing as correct.

The second is *adversarial*. An output deliberately induced by a malicious prompt, a poisoned retrieval source, or a manipulated tool. The most practically important variant for systems in production is *indirect prompt injection*: instructions embedded not in the user's prompt but in content the model is asked to process — retrieved documents, tool outputs, web

pages fetched by an agent. A retrieval pipeline that fetches arbitrary external content is processing potential adversarial inputs on every call. The model has no reliable mechanism to distinguish instructions from data; text that looks like a document to the system designer may look like a prompt to the model. These failures cross trust boundaries and require defenses in depth — input sanitization, output schema enforcement, privilege separation between retrieval and execution — not diagnostic ladders.

The third is *drift*. An output that was correct yesterday and is wrong today because the world has changed. This is a failure of the system’s integration with reality, not of any single output. The remedy lives in monitoring, evaluation, and governance.

The fourth is *complacency*. The five-axis procedure assumes a reviewer who is actively interrogating the output. It does not survive a reviewer who has stopped. A reviewer who has processed a long run of correct outputs will process the next one more shallowly; human-factors research on automation documents this systematically. The system records whether review happened, not how carefully. The model’s failure rate may be constant; the effective error rate seen by users rises as vigilance decays. The ladder can only be climbed by someone who is still climbing.

These four classes require different remediation than the ladder provides: governance and value alignment for normative failures; security engineering for adversarial failures; monitoring and refresh cycles for drift; sustained reviewer discipline for complacency. Hold the axes loosely: they are not orthogonal, failures often span multiple, and the ordering is a heuristic, not a theorem.

Walking the ladder

The five axes form a ladder because they are arranged in roughly increasing distance from what the LLM can reliably do on its own. The ladder is a procedure for the five-axis failures — the class most commonly encountered in practice and most directly remedied by prompt, system design, and process changes. The recommended walk, when an output arrives that strikes you as wrong, is bottom-up:

1. *Structure*: does the output parse, compile, validate against its schema?
2. *Meaning*: read it as a self-contained artifact — does it contradict itself?
3. *Context*: is it appropriate for the situation, audience, and purpose?
4. *Groundedness*: are the factual claims true?
5. *Environment*: was the question itself fully specified?

The discipline is that lower-axis failures are typically easier to diagnose and cheaper to remedy than higher-axis ones. A practitioner who jumps straight to “the model hallucinated” may have missed a context failure that does not require any change to the model at all. Conversely, a practitioner who never considers groundedness will deploy systems that are confidently wrong about the world.

For tasks that chain multiple inference steps — a function calling helpers, a pipeline transforming through several stages, an agent loop accumulating decisions — apply the ladder at each intermediate output, not only at the final result. A chain where every individual step passes the five-axis walk is not therefore correct; errors from early steps propagate as premises into later ones. The remedy is checkpointing and intermediate verification, not

per-step prompt improvement.

Return to the developer with the customer-records function. We walk the ladder.

Structure passes: the function parses and imports. *Meaning* passes: it does not contradict itself; the type signature matches the return. *Context* fails: the codebase uses a particular ORM and the model wrote raw list comprehensions instead. We add the codebase conventions to the prompt and the model writes ORM code.

The function still produces wrong output. We walk again. Structure, meaning, context now all pass. *Groundedness* fails: the model called `customers.filter_by_region(r)`, a method that looked plausible but does not exist on the actual ORM. We give the model the ORM definitions, by retrieval, and it picks a real method.

The function still produces wrong output. The methods are real. The output is sensibly wrong: the regions are off. *Environment* fails: in this codebase, “region” is a sales territory id, not a country code. The user assumed the model could see what they meant by region. It could not. The user specifies what region means, and the function works.

The walk shows the discipline. We did not stop at the first axis that passed. We did not jump to the most exotic explanation. We located the failure at the level where the remedy lives, and each fix targeted the right intervention point: prompt content for context, retrieval for groundedness, prompt specification for environment.

It is worth pausing on the groundedness failure to name the mechanism behind it. The invented method — `customers.filter_by_region(r)` — is a product of the same training objective that makes the model fluent in the first place. Next-token prediction trains on plausibility: method calls of the form `object.verb_by_field()` are common patterns in the training corpus, and the model produces what is statistically likely given the surrounding code. There is no signal in the pretraining objective that distinguishes a method that exists in the user’s actual ORM from one that merely fits the pattern. Notice also *where* the failure lands: not in the high-level structure of the function (filter, sort by numeric field, return names) but in a specific peripheral detail (the exact method name). This is characteristic of groundedness failures. The skeleton of a correct answer is more strongly represented in training data than the specific, deployment-relevant details that make the skeleton work. Hallucinated citations, invented API endpoints, and fabricated function signatures all follow the same shape: the genre is right, the specific referent is not. The remedy — giving the model the ORM schema before generation — works by changing the question from “what is statistically likely” to “what is consistent with this document.”

Why this matters

The pedagogical use of the model is twofold. For users, it is a diagnostic ladder. For engineers, it is a translation layer between the technical vocabulary of the field — cross-entropy, retrieval, alignment, RLHF — and the user-facing vocabulary of “the model got it wrong.” A complaint that arrives as “the bot hallucinated” can, after a quick walk through the axes, become “the prompt did not say what region meant, and the model assumed wrong.” The fix is no longer in the model and the conversation moves where it belongs.

The model also gives engineering teams a structured way to reason about where to spend their effort. Lower axes call for model-side and decoder-side fixes. Higher axes call for system-side fixes — retrieval, tool use, prompt-construction discipline, agent-context accumulation. A team that wants to improve groundedness will primarily work on retrieval, with model-side levers — factuality fine-tuning, calibrated abstention, citation-required

generation — as a smaller second front. A team that wants to improve environment will primarily work on its prompt-construction discipline rather than on retrieval. Knowing which axis you are on tells you which part of the playbook to consult.

The ladder also tells teams where to concentrate verification effort. AI generation has become cheap; careful verification has not fallen at the same rate. A team performing full five-axis review on every output is spending most of its attention above the groundedness axis, because structure and meaning are faster to clear. Knowing which axis is most likely to fail for a given task class lets that team concentrate its review where the risk actually is. The ladder is not only a diagnostic framework for failures that have already occurred; it is a guide to where to invest verification capacity before they do.

This is what the ladder is for. It is not a theory of language. It is a procedure for the moment in which an output is wrong and someone needs to say, in plain English, why.

References

- [1] S. Harnad, “The Symbol Grounding Problem,” *Physica D: Nonlinear Phenomena*, vol. 42, no. 1–3, pp. 335–346, 1990.
- [2] E. M. Bender and A. Koller, “Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 5185–5198.
- [3] Z. Ji *et al.*, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

Lodewijk Bonebakker is the author of Large Language Models as Engineered Systems, a forthcoming primer for engineers on the systems, statistical, and human-factors dimensions of modern LLMs. He can be reached at focus2flow@bonebakker.org.